MANAGING TERRAFORM STATE AND MULTI-ENVIRONMENT DEPLOYMENTS

DEPENDENCIES

Terraform uses dependencies to determine the order in which resources should be created, modified, or destroyed.

TYPES OF DEPENDENCIES IN TERRAFORM

1. Implicit Dependency
2. Explicit Dependency

IMPLICIT DEPENDENCY

1. implicit basically means terraform should decide which resources are created first
2. Terraform analyzes the code and decides the creation order without requiring any additional configuration.

1.sg.tf

```
data "aws_vpc" "default" {
  default = true
}

resource "aws_security_group" "sg" {
  name = "my-security-group"
  vpc_id = data.aws_vpc.default.id

  ingress {
    from_port = 80
    to_port = 80
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port = 0
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "my-security-group"
  }
```

```
}
```

2.ec2.tf

```
resource "aws_instance" "ec2" {
  ami                  = "ami-0685bcc683dadb6b9"
  instance_type = "t3.micro"
  key_name       = "ubuntu"
  vpc_security_group_ids  = [aws_security_group.sg.id]
  tags = {
    Name = "ec2"
  }
}
```

1. git add .
2. git commit -m “implicit”
3. git push

ON UBUNTU-LOCAL

1. git clone https://github.com/mayurpandit25/terraform-b30.git
2. cd terraform-b30
3. terraform init
4. terraform apply –auto-approve
5. terraform destroy –auto-approve

NOTE:
The EC2 instance is using the Security Group ID. Terraform automatically understands that:

1. Security Group must be created first.
2. EC2 instance must be created after the Security Group.

EXPLICIT DEPENDENCY

1. it depends totally on the user. User decides which resources are created first
2. Use explicit dependencies when Terraform cannot automatically determine the relationship between resources.

1.ec2.tf

```
resource "aws_instance" "ec2" {
  ami                  = "ami-0685bcc683dadb6b9"
  instance_type   = "t3.micro"
  key_name         = "ubuntu"
  vpc_security_group_ids = [aws_security_group.sg.id]
```

```
  depends_on      = [aws_security_group.sg]
  tags      = {
    Name = "ec2"
  }
}
```

2.sg.tf

```
data "aws_vpc" "default" {
  default = true
}

resource "aws_security_group" "sg" {
  name   = "my-security-group"
  vpc_id = data.aws_vpc.default.id

  ingress {
    from_port = 80
    to_port      = 80
    protocol     = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port      = 0
    protocol     = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "my-security-group"
  }
}
```

1. git add .
2. git commit -m “implicit”
3. git push

ON UBUNTU-LOCAL

1. git clone https://github.com/mayurpandit25/terraform-b30.git
2. cd terraform-b30

3. terraform init
4. terraform apply –auto-approve
5. terraform destroy –auto-approve
